Reflection

With 3 months instead of 8 hours. I would build the parts I stubbed, in this order:

1. **The mapping assistant with a mapping memory.** That is where most production effort goes: every new ERP and entity brings unmapped accounts, and approved mappings should make the next period fully deterministic.
2. **The lineage store as a real table** (cell to account to ledger line to source row) with a drill-down UI. A CSV column works for a prototype, but auditors need to click through.
3. **Multi-entity consolidation:** IC matching across entities, eliminations, and translation with CTA.
4. **Proper calibration** of mapping and fix confidence against historical reviewer decisions, replacing my hand-set scores.
5. **A role model:** four-eyes approval, role permissions, and sign-offs that expire when the numbers underneath change.
6. **Regression tests on real anonymised closes,** not just seeded defects.

Where the prototype breaks at scale.

- **Speed:** posting verification recomputes every account against every entry, which is O(accounts x lines). That is fine for 60 accounts and slow for 50,000 GL lines; it needs indexed aggregation.
- **Memory:** everything is loaded at once.
- **Fix generation is per-entry and naive:** name similarity uses token overlap with a hand-written synonym table, which will not survive five ERPs and three languages.
- **Single entity, single period:** consolidation needs entity as a first-class key on every line, IC matching as a graph problem, and per-entity FX policy.
- **File-based overrides:** a JSON file has no concurrency and no permissions.
- **Template explanations repeat themselves** across thousands of entries. Reviewers need them grouped ("37 entries hit unmapped accounts in the 63xx range").

How I used AI tools. I used Claude as an agentic pair programmer through the whole task.

- **Where it helped:** it wrote a probe script that found most of the defects in minutes, including the ones the PDF does not list (the prior TB is out of balance, retained earnings does not roll, JE-003 matches no rate). It also drafted most of the boilerplate (loaders, writers, CLI) and the first version of the tests.
- **Where I had to correct it:**
    - The first version of the fix generator said "every option has been re-checked against the rules" in the report, but the balance repairs for JE-002 were never re-validated. A test caught it, and re-validation was moved to one choke point that every candidate has to pass.
    - The obvious nearest mapping for 6315 is 6310, which would have turned JE-005 into a no-op entry. I made sure that case is discarded and shown to the reviewer rather than offered.
    - It treated "unbalanced after FX rounding" at face value. The numbers say the gap is 182k, not rounding, so I turned that into a clarifying question.
    - It produced output that was not deterministic (set ordering in an account label), which would have broken the idempotency test.
- **What I kept for myself:** I made the accounting calls: what counts as fatal vs fixable, which rules can never be overridden, and which questions to ask. I did not let the tool decide those.

What I think is being underestimated. The trial balance is not enough input. A TB gives closing balances, but three of the four statements need *movements*:

- **Cash flow:** capex vs non-cash additions (PP&E rose by 5.2M with no detail), disposals, the FX effect on cash, and debt drawdowns vs reclasses.
- **SOCIE:** dividends, share issues and buybacks, and OCI by component.
- **FX:** functional-currency book values for foreign rows, without which you cannot even prove the TB balances.

Without that, an "AI-generated cash flow statement" will be confidently wrong, and the LLM will hide it by making the numbers sound reasonable. The real product question is the **data contract with the ERP**: which subledger extracts are mandatory, and what the system refuses to produce when they are missing. I would make "insufficient input to produce this statement" an output the system is designed to give, not a failure.